SOFTWARE ENGINEERING

Chapter 3: Agility and Process

Based on: Software Engineering: A Practitioner's Approach (9th Edition)

Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri
Instructor: Engineer Rasoulzadeh
Course: Software Engineering

1. Introduction

Software development is a complex activity that requires planning, communication, design, coding, testing, and maintenance. Traditional software development methods often follow a fixed sequence of activities. However, modern software projects frequently face changing requirements and customer expectations.

Agile software development provides an approach that helps development teams respond to change, deliver software faster, and maintain continuous communication with customers.

This chapter introduces agility, agile principles, agile processes, and popular agile methods such as Extreme Programming (XP) and Scrum.

2. What Is Agility?

Agility is the ability of a software development team to respond quickly and effectively to changes.

In software engineering, changes may occur because of:

- New customer requirements.
- Changes in business conditions.
- New technologies.
- Competition in the market.
- Feedback from users.

An agile team does not assume that all requirements will remain unchanged throughout a project. Instead, it develops software in small steps and improves the product continuously.

Example:

Imagine a company wants to develop an online shopping application. At first, the customer requests basic features such as product browsing and shopping carts. After seeing the first version, the customer requests online payment and order tracking.

An agile team can add these features in later development cycles instead of redesigning the entire project.

3. The Agile Manifesto

The Agile Manifesto introduces four important values that guide agile software development.

3.1 Individuals and Interactions Over Processes and Tools

Communication and teamwork are more important than relying only on tools and procedures.

Example: Developers, designers, and testers communicate directly to solve a problem instead of waiting for long formal reports.

3.2 Working Software Over Comprehensive Documentation

Documentation is useful, but delivering software that works is the main priority.

Example: A team delivers a working login feature early so that users can test it and provide feedback.

3.3 Customer Collaboration Over Contract Negotiation

Customers should participate throughout development, not only at the beginning or end.

Example: The customer attends regular review meetings and explains whether the application meets business needs.

3.4 Responding to Change Over Following a Plan

A development plan is important, but the team must be able to adapt when requirements change.

Example: If customers request a new payment method, the team can prioritize it in an upcoming development cycle.

4. Principles of Agile Software Development

The Agile Manifesto is supported by twelve principles. Important ideas include:

1. Satisfy customers through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently.
4. Encourage daily cooperation between business people and developers.
5. Build projects around motivated individuals.
6. Prefer direct communication whenever practical.
7. Measure progress mainly through working software.
8. Maintain a sustainable development pace.
9. Pay continuous attention to technical excellence and good design.
10. Keep things simple.
11. Encourage self-organizing teams.
12. Regularly reflect on how to become more effective.

Example:

A team developing a university registration system releases a basic course-registration feature first. Students and staff test it and suggest improvements. The team uses this feedback to improve the next release.

5. Agile Process

An agile process develops software through repeated cycles. Each cycle includes activities such as planning, design, implementation, testing, and feedback.

Instead of delivering the entire system at the end, the team delivers smaller working parts throughout the project.

5.1 Iterative and Incremental Development

Iterative development means improving a product through repeated cycles.

Incremental development means adding new features to the product step by step.

Example: Online Shopping Application

- Increment 1: Product catalogue.
- Increment 2: User registration and login.
- Increment 3: Shopping cart.
- Increment 4: Online payment.
- Increment 5: Order tracking.

At the end of each increment, the application has additional working functionality.

6. Extreme Programming (XP)

Extreme Programming (XP) is an agile software development method that emphasizes customer involvement, frequent releases, simple design, and continuous improvement.

Important XP practices include:

6.1 Planning Game

Developers and customers work together to decide which features should be developed first.

Example: The customer identifies product search as more important than product reviews, so the team prioritizes search.

6.2 Small Releases

The team releases working software frequently.

Example: A basic version of the shopping application is released before all features are completed.

6.3 Simple Design

The team creates the simplest design that meets current requirements.

Example: The developers build a straightforward shopping cart rather than adding unnecessary complex features.

6.4 Test-Driven Development (TDD)

Developers write tests before writing the corresponding production code.

The general cycle is:

1. Write a test that initially fails.
2. Write code to make the test pass.
3. Refactor the code while keeping the tests passing.

6.5 Refactoring

Refactoring means improving the internal structure of code without changing its external behavior.

Example: A developer reorganizes repeated payment-related code into reusable functions.

6.6 Pair Programming

Two developers work together at one workstation. One writes code while the other reviews and thinks about improvements. They can switch roles regularly.

6.7 Continuous Integration

Developers frequently integrate their code into a shared project and run automated tests to detect problems early.

6.8 Collective Code Ownership

Team members share responsibility for the code. Any qualified team member can improve or correct a part of the system.

6.9 On-Site Customer

A customer representative remains available to answer questions and clarify requirements during development.

7. Scrum Framework

Scrum is a popular framework for managing agile software development. It organizes work into short development periods called Sprints.

A Sprint is a fixed-length period, commonly one to four weeks, during which the team works toward a specific goal.

7.1 Scrum Roles

Product Owner: Responsible for maximizing product value and managing the Product Backlog.

Scrum Master: Helps the team understand and use Scrum effectively and works to remove obstacles.

Developers: The professionals who create the product increment during each Sprint.

7.2 Scrum Artifacts

Product Backlog: An ordered list of work needed for the product.

Sprint Backlog: The selected Product Backlog items and the team's plan for completing them during the Sprint.

Increment: The usable result produced during a Sprint, which meets the agreed quality requirements.

7.3 Scrum Events

Sprint Planning: The team plans the work and defines the Sprint Goal.
Daily Scrum: A short daily event for Developers to inspect progress and adapt their plan.
Sprint Review: The team and stakeholders inspect the outcome and discuss future changes.
Sprint Retrospective: The team reflects on its process and identifies improvements.

8. Agile Planning and User Stories

Agile planning is flexible. Teams regularly review priorities and adjust plans according to customer feedback and project progress.

A common way to describe requirements is through user stories.

A user story usually follows this format:

> *"As a [type of user], I want [a feature], so that [a benefit]."*

Example:

> *"As a customer, I want to add products to my shopping cart so that I can purchase several products together."*
> *"As a customer, I want to track my order so that I can know when it will arrive."*

User stories help teams focus on the value a feature provides to the user.

9. Agile Testing and Continuous Feedback

Testing is an ongoing activity in agile development. It should not be postponed until the entire project is finished.

Agile teams use different testing activities, including:

- Unit testing: Testing individual functions or components.
- Integration testing: Checking whether different components work together.
- System testing: Testing the complete application.
- Acceptance testing: Checking whether the product meets customer expectations.
- Regression testing: Ensuring that new changes have not broken existing features.

Example:

When developers add an online payment feature, they test successful payments, rejected payments, incorrect card details, and network failures.

Customer feedback also helps the team identify problems and improve the next version.

10. Advantages of Agile Development

Agile development offers several advantages:

1. Flexibility: Teams can respond to changing requirements.
2. Customer Satisfaction: Customers participate throughout development.
3. Early Delivery: Useful features can be released sooner.
4. Better Communication: Frequent collaboration improves teamwork.

5. Continuous Improvement: Regular reviews help identify problems.
6. Reduced Risk: Problems can be discovered earlier through frequent testing and feedback.

11. Challenges of Agile Development

Despite its advantages, agile development also has some challenges:

1. Customer Availability: Continuous customer participation may be difficult.
2. Changing Requirements: Frequent changes can affect priorities and schedules.
3. Team Experience: Agile methods require collaboration and disciplined teamwork.
4. Documentation: If documentation is neglected, maintenance may become difficult.
5. Project Estimation: Predicting the exact cost and completion date can be challenging.
6. Large Projects: Coordinating several agile teams may require additional planning.

Agile is not automatically the best choice for every project. The development approach should be selected according to the project's needs, risks, team, and environment.

12. Practical Example: Developing an Online Shopping Application Using Agile

12.1 Project Description

A company wants to create an online shopping application where customers can browse products, create accounts, add products to their shopping carts, make payments, and track their orders.

The development team decides to use Scrum.

12.2 Product Backlog

The Product Owner creates the following initial Product Backlog:

ID	User Story	Priority
1	As a customer, I want to browse products.	High
2	As a customer, I want to register and log in.	High
3	As a customer, I want to add products to my cart.	High
4	As a customer, I want to pay online.	High
5	As a customer, I want to track my orders.	Medium
6	As a customer, I want to review products.	Low

12.3 Sprint Plan

The team divides the work into five Sprints.

Sprint 1: Product Catalogue

- Design the product page.
- Display product names and prices.
- Add product categories.

- Test product browsing.

Sprint 2: User Accounts

- Create registration functionality.
- Implement login and logout.
- Validate user information.
- Test account security.

Sprint 3: Shopping Cart

- Add products to the cart.
- Remove products from the cart.
- Calculate the total price.
- Test cart operations.

Sprint 4: Payment and Orders

- Integrate a payment service.
- Process successful and failed payments.
- Create order records.
- Test payment scenarios.

Sprint 5: Order Tracking

- Display order status.
- Show estimated delivery information.
- Improve the user interface.
- Perform final acceptance testing.

12.4 Example of a Daily Scrum

During a Daily Scrum, three developers discuss their progress:

Developer 1: Yesterday, I completed the product catalogue. Today, I will improve the product search feature.

Developer 2: Yesterday, I worked on the shopping cart. Today, I will implement the total price calculation.

Developer 3: Yesterday, I tested the registration page. Today, I will fix the validation errors.

The team also discusses any obstacles that may prevent progress.

12.5 Expected Project Result

At the end of the five Sprints, the team expects to deliver an online shopping application with:

- Product browsing and search.
- Customer registration and login.
- Shopping cart functionality.
- Online payment.
- Order management and tracking.

The application is developed incrementally, and feedback can be used to improve it throughout the process.

13. When Is Agile Suitable?

Agile development is particularly useful when:

- Requirements may change frequently.
- Customer feedback is important.
- The project can be divided into smaller releases.
- The development team can communicate regularly.
- Early delivery of useful features is valuable.

For projects with strict regulatory requirements, fixed contracts, or highly critical systems, additional planning, documentation, verification, and control may be necessary.

14. Conclusion

Agility is an important concept in modern software engineering. It helps development teams respond to change, communicate with customers, and deliver working software in small increments.

The Agile Manifesto provides four core values and twelve principles that guide agile development. Methods such as Extreme Programming and frameworks such as Scrum offer practical ways to apply these ideas.

The online shopping application example demonstrates how agile development can organize work into manageable Sprints and deliver useful features step by step.

In conclusion, agile software development can improve flexibility, collaboration, and customer satisfaction when it is applied appropriately and supported by a skilled and committed team.

15. References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Beck, K., et al. (2001). *Manifesto for Agile Software Development*. https://agilemanifesto.org/

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/